Zaawansowane techniki programowania

Laboratorium 5

ZMIENDAK WIKTOR

ZADANIA OBOWIĄZKOWE:

1. Uzupełniamy klasę InMemoryMessageBroker w taki sposób, aby wiadomość trafiała do subskrybentów z tym samym routingKey, a sama publikacja była asymetryczna.

```java
@Override
public void publish(String routingKey, Object message) {
  List<Consumer<Object>> handlers = subscribers.get(routingKey);
  if (handlers == null || handlers.isEmpty()) {
    return;
  }

  for (Consumer<Object> handler : handlers) {
    executor.execute(() -> handler.accept(message));
  }
}
```

2. W klasie OrderEventPublisher publikujemy event routingKey orders.created.

```java
public void publishOrderCreated(OrderCreatedEvent event) {
  broker.publish(routingKey: "orders.created", event);
}
```

3. W OrderService budujemy klasę odpowiedzialną za tworzenie eventu oraz wywołujemy publisher.publishOrderCreated.

```java
public Order create(CreateOrderRequest req) {
  Product p = catalog.getProducts().get(req.getProductId());
  if (p == null) throw new ResponseStatusException(HttpStatus.NOT_FOUND, reason: "Product not found");

  double total = p.getPrice() * req.getQuantity();
  long id = orderStore.nextId();
  Order order = new Order(id, req.getProductId(), req.getQuantity(), total);
  orderStore.getOrders().put(id, order);

  OrderCreatedEvent event = new OrderCreatedEvent(UUID.randomUUID().toString(), id, req.getProductId(), req.getQuantity(

  publisher.publishOrderCreated(event);

  return order;
}
```

4. Uzupełniamy OrderCreatedListener tak aby po odebraniu wiadomości rzutowano ją na OrderCreatedEvent.

```
public OrderCreatedListener(MessageBroker broker, ReceivedEventsStore store) {
  this.broker = broker;
  this.store = store;
}

@PostConstruct
public void init() {
  broker.subscribe(routingKey: "orders.created", message -> {
    if (message instanceof OrderCreatedEvent event) {
      store.addIfFirstTime(event);
    }
  });
}
```

ZADANIA DODATKOWE:

1. Rozszerzamy walidację eventid.

```
public boolean addIfFirstTime(OrderCreatedEvent e) {
  if (e == null || e.getEventId() == null || e.getEventId().isBlank()) return false;
```

2. Przy okazji obsługujemy zdarzenie w którym dwa różne event są wpisywane osobno w store.
3. Dodatkowo jeżeli eventid jest nullem to jest wtedy ignorowany.
4. Zapisujemy tylko przy pierwszym evencie.

```
    if (first)
      events.add(e);
```

WYNIKI TESTÓW:

```
2026-03-31T21:40:09.826+02:00  INFO 26628 --- [           main] o.s.t.web.servlet.TestDispatcherServlet  : Initializing Servlet ''
2026-03-31T21:40:09.828+02:00  INFO 26628 --- [           main] o.s.t.web.servlet.TestDispatcherServlet  : Completed initialization
 in 2 ms
2026-03-31T21:40:09.885+02:00  INFO 26628 --- [           main] p.e.lab5.Grade3PublishAndConsumeTest     : Started Grade3PublishAnd
ConsumeTest in 2.708 seconds (process running for 5.255)
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.957 s -- in pl.example.lab5.Grade3PublishAndConsumeTest
[INFO] Running pl.example.lab5.Grade3PublisherRoutingKeyTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.025 s -- in pl.example.lab5.Grade3PublisherRoutingKeyTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  8.018 s
[INFO] Finished at: 2026-03-31T21:40:10+02:00
[INFO] ------------------------------------------------------------------------
```

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.066 s -- in pl.example.lab5.Grade4MultipleDistinctEvents
st
[INFO] Running pl.example.lab5.Grade4NullEventIdIgnoredTest
2026-03-31T21:50:56.504+02:00  INFO 22800 --- [           main] t.c.s.AnnotationConfigContextLoaderUtils : Could not detect defau
 configuration classes for test class [pl.example.lab5.Grade4NullEventIdIgnoredTest]: Grade4NullEventIdIgnoredTest does not decla
 any static, non-private, non-final, nested classes annotated with @Configuration.
2026-03-31T21:50:56.507+02:00  INFO 22800 --- [           main] .b.t.c.SpringBootTestContextBootstrapper : Found @SpringBootConfi
ration pl.example.lab5.Lab5Application for test class pl.example.lab5.Grade4NullEventIdIgnoredTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.206 s -- in pl.example.lab5.Grade4NullEventIdIgnoredTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  8.067 s
[INFO] Finished at: 2026-03-31T21:50:56+02:00
[INFO] ------------------------------------------------------------------------
```